CS3500 Operating Systems
Lab 1: Utilities

Due: 11:59PM on 13 August 2026

Setting up

1. Clone the `xv6-labs` repo:

   ```
   git clone https://github.com/nandhagk/xv6-labs
   ```

2. Install Docker by following the instructions for your platform.
   - Windows: [Docker Desktop]
   - MacOS: [Docker Desktop]
   - Linux: [Docker Engine]

3. Once Docker is installed, pull the `xv6-tools` Docker image using:

   ```
   docker pull nandhagk/xv6-tools:latest
   ```

4. To run the repo inside Docker use:

   ```
   docker run -it -v <path to xv6-labs>/xv6-labs:/home/xv6-labs nandhagk/xv6-tools:latest
   ```

   ```
   -i → Makes the container interactive
   -t → Sets up a terminal to interact with the container
   -v → Mounts a volume into the container
   ```

   For the following directory structure, `<path to xv6-labs>` is `/home/nandhagk/Projects`.

   ```
   📁 /home/nandhagk/Projects/
     └ 📁 xv6-labs/
         ├ 📁 conf/
         ├ 📁 kernel/
         └ ...
   ```

   At this point, you should observe a bash prompt

   ```
   bash-5.3#
   ```

5. Build and run `xv6` using:

   ```
   bash-5.3# cd /home/xv6-labs && make qemu
   ```

   You should observe the following

   ```
   ...
   balloc: first 969 blocks have been allocated
   balloc: write bitmap block at sector 46
   qemu-system-riscv64 -machine virt -bios none -kernel kernel ...

   xv6 kernel is booting

   hart 1 starting
   hart 2 starting
   init: starting sh
   $
   ```

6. To quit `qemu` type `Ctrl`+`a` `x` (press `Ctrl` and `a` at the same time, followed by `x`).

`sleep`

This exercise makes you familiar with writing a user program on xv6 and the `pause` system call.

Implement a user-level `sleep` program for xv6, along the lines of the UNIX `sleep` command. Your `sleep` should pause for a user-specified number of ticks. A tick is a notion of time defined by the xv6 kernel, namely the time between two interrupts from the timer chip. Your solution should be in the file `user/sleep.c`.

Some hints:
- Before you start coding, read Chapter 1 of the xv6 book.
- Put your code in `user/sleep.c`. Look at some of the other programs in `user/` (e.g., `user/echo.c`, `user/grep.c`, and `user/rm.c`) to see how command-line arguments are passed to a program.
- Add your `sleep` program to `UPROGS` in the `Makefile`; once you've done that, `make qemu` will compile your program and you'll be able to run it from the xv6 shell.
- If the user forgets to pass an argument, `sleep` should print an error message.
- The command-line argument is passed as a string; you can convert it to an integer using `atoi` (see `user/ulib.c`).
- Use the system call `pause()`.
- See `kernel/sysproc.c` for the xv6 kernel code that implements the `pause()` system call (look for `sys_pause`), `user/user.h` for the C definition of `pause()` callable from a user program, and `user/usys.S` for the assembler code that jumps from user code into the kernel for `pause()`.

Run the program from the xv6 shell:

```
$ make qemu
...
init: starting sh
$ sleep 100
(nothing happens for a little while)
$
```

Your program should pause when run as shown above. Run `make grade` in your command line (outside of `qemu`) to see if you pass the sleep tests.

Note that `make grade` runs all tests, including the ones for the tasks below. If you want to run the grade tests for one task, type:

```
$ ./grade-lab-util sleep
```

This will run the grade tests that match `sleep`. Or, you can type:

```
$ make GRADEFLAGS=sleep grade
```

which does the same.

`sixfive`

In this exercise you'll use the system calls `open` and `read`, C strings, and processing text files in C.

For each input file, `sixfive` must print all the numbers in the file that are multiples of 5 or 6. Numbers are a sequence of decimal digits separated by characters in the string `" -\r\t\n./,"`. Thus, for the six in `xv6` `sixfive` shouldn't print `6` but, for example, `/6,` it should.

The following example illustrates `sixfive`'s behavior:

```
$ sixfive sixfive.txt
5
100
18
6
$
```

Some hints:
- Read the input file a character at a time.
- You can test if a character matches any of the separators using `strchr` (see `user/ulib.c`).
- Start and end of file are implicit separators.

`memdump`

This exercise will give you more practice using C pointers.

Have a look at `user/memdump.c`. Your job is to implement the function `memdump(char *fmt, char *data, int len)`. `memdump()`'s purpose is to print the contents of the `len` valid bytes of memory pointed to by `data` in the format described by the `fmt` argument. The format is a C string. Each character of the string indicates how to print successive parts of the data. Thus, for example, a C struct with multiple fields can be printed with a format string containing multiple characters.

Your `memdump()` should handle the following format characters:
- `i`: print the next 4 bytes of the data as a 32-bit integer, in decimal.
- `p`: print the next 8 bytes of the data as a 64-bit integer, in hex.
- `h`: print the next 2 bytes of the data as a 16-bit integer, in decimal.
- `c`: print the next 1 byte of the data as an 8-bit ASCII character.
- `s`: the next 8 bytes of the data contain a 64-bit pointer to a C string; print the string.
- `S`: the rest of the data contains the bytes of a null-terminated C string; print the string.

`memdump()` must respect the `len` argument and never read past the `len` valid bytes of data. If a format character would require more bytes than remain (for example an `i` that needs 4 bytes when only 2 are left), `memdump()` must print

```
memdump: not enough data for 'X'
```

where `X` is that format character, and then stop. For the `S` format, print the string up to its terminating null byte or the end of the valid data, whichever comes first.

Feel free to use C's `printf()` in your `memdump()`.

The `memdump` program, if executed with no arguments, calls `memdump()` with some example format strings and data. If `memdump()` is correctly implemented, the output will be:

```
$ memdump
Example 1:
61810
2026
Example 2:
a string
Example 3:
another
Example 4:
BD0
1819438967
100
z
xyzzy
Example 5:
hello
w
o
r
l
d
```

You will likely get a different hex address for the first line of the Example 4 output.

If the `memdump` program is invoked with an argument, it will read its standard input up to an end of file, and then call `memdump()` with the format and input data. So, once `memdump()` is implemented:

```
$ echo deadc0de | memdump hhcccc
25956
25697
c
0
d
e
$ echo deadc0de | memdump p
6564306364616564
$
```

The bounds check fires when the input is too short for a format character. Here `echo a` supplies only two bytes (`'a'` and a newline), but `i` needs four:

```
$ echo a | memdump i
memdump: not enough data for 'i'
$
```

Implement `memdump()`.

`find`

This exercise explores pathnames and directories, and the system calls `open`, `read`, and `fstat`.

Write a simple version of the UNIX `find` program for xv6: find all the files in a directory tree with a specific name. Your solution should be in the file `user/find.c`.

Some hints:
- Look at `user/ls.c` to see how to read a directory.
- Use recursion to allow `find` to descend into sub-directories.
- Don't recurse into `.` and `..`.
- Each time you invoke `make qemu`, it will build a new `fs.img`, removing files created in a previous run. If you would like to start qemu with the file system from a previous use, use `make qemu-fs`.
- Note that == does not compare strings as in Python. Use `strcmp()` instead.
- Add the program to `UPROGS` in the `Makefile`.

Your solution should produce the following output (when the file system contains the files `b`, `a/b` and `a/aa/b`):

```
$ make qemu
...
init: starting sh
$ echo > b
$ mkdir a
$ echo > a/b
$ mkdir a/aa
$ echo > a/aa/b
$ find . b
./b
./a/b
./a/aa/b
$
```

Run `make grade` to see what our tests think.

`exec`

This exercise involves the system calls `fork`, `exec`, and `wait`.

Add a `-exec cmd` option to `find`, which executes the program `cmd file` for each file `f` that `find` finds, instead of printing matching file names.

The following example illustrates `find -exec` behavior:

```
$ find . wc -exec echo hi
hi ./wc
$
```

Note that the command here is `echo hi` and the file is `./wc`, making the command `echo hi ./wc`, which outputs `hi ./wc`.

Some hints:
- Use `fork` and `exec` to invoke the command on each file. Use `wait` in the parent to wait for the child to complete the command.
- `kernel/param.h` declares `MAXARG`, which may be useful if you need to declare an `argv` array.

To test your solution for `find`, run the shell script `findtest.sh`. Your solution should produce the following output:

```
$ make qemu
...
init: starting sh
$ sh < findtest.sh
$ echo DONE
$ $ $ $ $ hello
hello
hello
$ $
```

The output has many `$` because the xv6 shell doesn't realize it is processing commands from a file instead of from the console, and prints a `$` for each command in the file.

Submitting the lab

Time spent

Create a new file, `time.txt`, and put in a single integer, the number of hours you spent on the lab. `git add` and `git commit` the file.

References

Create a new file, `references.txt`, and state who you collaborated with and which resources, possibly on the Internet, that you consulted. You must be as thorough and complete as possible. You must provide clear and specific attribution to any external sources that were used (e.g., "software library X at https://site.com was used to implement feature Y in the submitted program", or genAI service "X" with prompts "A","B","C" used to fix issue "W"). Consult the [Course Webpage] for more information. `git add` and `git commit` the file.

Submit

Assignment submissions are handled by Moodle.

When you're ready to submit, run `make zipball`, which will generate `lab.zip`. Upload this zip file to the corresponding Moodle assignment.

If you run `make zipball` and you have either uncommitted changes or untracked files, you will see output similar to the following:

```
 M hello.c
?? bar.c
?? foo.pyc
Untracked files will not be handed in.  Continue? [y/N]
```

Inspect the above lines and make sure all files that your lab solution needs are tracked, i.e., not listed in a line that begins with `??`. You can cause git to track a new file that you create using `git add {filename}`.

- Please run `make grade` to ensure that your code passes all of the tests.
- Commit any modified source code before running `make zipball`.

Optional challenge exercises

- Write an `uptime` program that prints the uptime in terms of ticks using the `uptime` system call.
- Support regular expressions in name matching for `find`. `grep.c` has some primitive support for regular expressions.
- The xv6 shell (`user/sh.c`) is just another user program. It lacks many features found in real shells, but you can modify and improve it. For example,
  - Modify the shell to not print a `$` when processing shell commands from a file
  - Modify the shell to support `wait`
  - Modify the shell to support tab completion
  - Modify the shell to keep a history of passed shell commands, or anything else you would like your shell to do. (If you are very ambitious, you may have to modify the kernel to support the kernel features you need; xv6 doesn't support much.)

Acknowledgements

A significant portion of this lab assignment is derived from `https://pdos.csail.mit.edu/6.828/2026/labs/util.html`.